

Week-5

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Ans: Limitations

- Disk based processing.
- Slow for iterative tasks.
- High latency.
- Complex coding.
- No real-time processing.

Why Spark?

- In-memory processing.
- Faster execution.
- Supports ML and Streaming.
- Easy APIs.
- Fault tolerant.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Ans: Spark stores intermediate data in RAM instead of disk, making machine learning and iterative algorithms much faster.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Ans: `df_no_duplicates = df.dropDuplicates(["user_id","transaction_date"])`

`df_no_duplicates.show()`

user_id	transaction_date	region	product_category	sale_amount	age	subscription	city	status	raw_timestamp	email	username	store_id
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101
2	2025-01-02	East	Clothing	300	22	Premium	Mumbai	NULL	2025-01-02 11:00:00	user2@gmail.com	Alice	102
3	2025-01-03	West	Electronics	700	29	Basic	Delhi	Inactive	2025-01-03 09:30:00	NULL	Bob	101
4	2025-01-04	South	Furniture	NULL	35	Premium	Chennai	Active	2025-01-04 08:00:00	user4@gmail.com		103
5	2025-01-05	West	Clothing	450	19	Premium	Delhi	NULL	2025-01-05 12:00:00	user5@gmail.com	David	102

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Ans: from pyspark.sql.functions import avg

```
west_sales = df.filter(df.region=="West")\
.groupBy("product_category")\
.agg(avg("sale_amount").alias("Average_Sales"))
west_sales.show()
```

product_category	Average_Sales
Electronics	566.6666666666666
Clothing	450.0

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Ans: 1. Removes rows containing null values.

```
df.na.drop().show()
```

user_id	transaction_date	region	product_category	sale_amount	age	subscription	city	status	raw_timestamp	email	username	store_id
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101

2. Replaces null values.

```
df_status = df.na.fill({"status":"Unknown"})
df_status.show()
```

user_id	transaction_date	region	product_category	sale_amount	age	subscription	city	status	raw_timestamp	email	username	store_id
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101
2	2025-01-02	East	Clothing	300	22	Premium	Mumbai	Unknown	2025-01-02 11:00:00	user2@gmail.com	Alice	102
3	2025-01-03	West	Electronics	700	29	Basic	Delhi	Inactive	2025-01-03 09:30:00		Bob	101
4	2025-01-04	South	Furniture	NULL	35	Premium	Chennai	Active	2025-01-04 08:00:00	user4@gmail.com		103
5	2025-01-05	West	Clothing	450	19	Premium	Delhi	Unknown	2025-01-05 12:00:00	user5@gmail.com	David	102
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Ans: from pyspark.sql.functions import count

```
city_count = df.groupBy("city")\
.agg(count("*").alias("Count"))\
.filter("Count > 100")

city_count.show()
```

```
+-----+-----+
|city|Count|
+-----+-----+
+-----+-----+
```

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Ans: new_df = df.drop("status")

```
new_df = new_df.withColumnRenamed(
    "sale_amount",
    "price"
)
```

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Ans: premium_users = df.filter(
 (df.age>=18) &
 (df.age<=30) &
 (df.subscription=="Premium")
)

```
premium_users.show()
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount|age|subscription|city|status|raw_timestamp|email|username|store_id|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1|2025-01-01|West|Electronics|500|25|Premium|Delhi|Active|2025-01-01 10:00:00|user1@gmail.com|John|101|
|2|2025-01-02|East|Clothing|300|22|Premium|Mumbai|NULL|2025-01-02 11:00:00|user2@gmail.com|Alice|102|
|5|2025-01-05|West|Clothing|450|19|Premium|Delhi|NULL|2025-01-05 12:00:00|user5@gmail.com|David|102|
|1|2025-01-01|West|Electronics|500|25|Premium|Delhi|Active|2025-01-01 10:00:00|user1@gmail.com|John|101|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Ans: Null values can produce incorrect averages and sums.

```
clean_df = df.na.fill({"sale_amount":0})
```

Benefits:

- Accurate average
- Correct sum
- Prevent errors

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

Ans: from pyspark.sql.functions import col

```
from pyspark.sql.types import TimestampType
```

```
df_time = df.withColumn(  
    "raw_timestamp",  
    col("raw_timestamp").cast(TimestampType())  
)
```

```
df_time = df_time.withColumnRenamed(  
    "raw_timestamp",  
    "event_time"  
)
```

```
df_time.show()
```

user_id	transaction_date	region	product_category	sale_amount	age	subscription	city	status	event_time	email	username	store_id
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101
2	2025-01-02	East	Clothing	300	22	Premium	Mumbai	NULL	2025-01-02 11:00:00	user2@gmail.com	Alice	102
3	2025-01-03	West	Electronics	700	29	Basic	Delhi	Inactive	2025-01-03 09:30:00	NULL	Bob	101
4	2025-01-04	South	Furniture	NULL	35	Premium	Chennai	Active	2025-01-04 08:00:00	user4@gmail.com		103
5	2025-01-05	West	Clothing	450	19	Premium	Delhi	NULL	2025-01-05 12:00:00	user5@gmail.com	David	102
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Ans: During groupBy Spark redistributes data among partitions.

```
df.groupBy("city").count()
```

```
DataFrame[city: string, count: bigint]
```

This causes shuffle.

Why Wide Transformation?

Because data moves between partitions.

Examples:

- `groupBy()`
- `join()`
- `distinct()`

Q12: Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

Ans: from pyspark.sql.functions import col

```
clean_users = df.filter(
    (col("email").isNull()) &
    (col("username") != ""))
)
clean_users.show()
```

user_id	transaction_date	region	product_category	sale_amount	age	subscription	city	status	raw_timestamp	email	username	store_id
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101
2	2025-01-02	East	Clothing	300	22	Premium	Mumbai	NULL	2025-01-02 11:00:00	user2@gmail.com	Alice	102
5	2025-01-05	West	Clothing	450	19	Premium	Delhi	NULL	2025-01-05 12:00:00	user5@gmail.com	David	102
1	2025-01-01	West	Electronics	500	25	Premium	Delhi	Active	2025-01-01 10:00:00	user1@gmail.com	John	101

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Ans: from pyspark.sql.functions import min,max,mean

```
df.agg(
```

```
min("sale_amount").alias("Minimum"),
max("sale_amount").alias("Maximum"),
mean("sale_amount").alias("Average")
).show()
```

```
+-----+-----+-----+
|Minimum|Maximum|Average|
+-----+-----+-----+
|    300|    700|  490.0|
+-----+-----+-----+
```

Q14: In the context of **cleaning a dataset**, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Ans: If dates are inconsistent:

```
01/05/2025
2025-01-05
5-Jan-2025
```

Spark may infer incorrect data types and generate null values or wrong schema.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by store_id to calculate total revenue.

Ans: from pyspark.sql.functions import sum

```
final_df = df.dropDuplicates()\
    .na.fill({"sale_amount":0})\
    .groupBy("store_id")\
    .agg(sum("sale_amount").alias("Total_Revenue"))
final_df.show()
```

store_id	Total_Revenue
103	0
101	1200
102	750

Brief Insights

Data Cleaning

- Removed duplicate records.
- Filled missing values.
- Deleted invalid rows.

Transformations

- Applied filtering.
- Renamed columns.
- Converted timestamps.

Aggregations

- Calculated count, sum, avg, min and max.
- Used groupBy for analysis.

Performance

- Spark performs computations in memory.
- groupBy introduces shuffle and is a wide transformation.
- Immutable DataFrames preserve original data.